

NODE 3-TIER APP

AWS ECS Fargate Deployment

Troubleshooting & Incident Documentation

Project: OniSamuelOpeyemi/3tiernode
Platform: AWS ECS Fargate + Aurora PostgreSQL
Region: us-east-1 (N. Virginia)
IaC: Terraform (modular)
Pipeline: GitHub Actions
Total Issues Resolved: 16

Overview

This document records every technical issue encountered during the deployment of the Node 3-tier application on AWS, including the error message, root cause, and resolution applied. It serves as both a post-mortem record and a reference for future deployments of similar architectures.

The architecture consists of:

- Web tier — ECS Fargate behind CloudFront + ALB in private subnets
- API tier — ECS Fargate behind CloudFront + ALB in private subnets
- Database tier — Aurora PostgreSQL in isolated DB subnets, no internet access
- CI/CD — GitHub Actions pipeline (test → security scan → build → deploy)
- IaC — Terraform with 6 modular components (networking, ECR, ECS, database, CDN, monitoring)

Summary of All Issues

#	Issue	Root Cause	Fix
1	Terraform syntax error	Semicolons in HCL blocks	Expand to multi-line blocks
2	Undeclared variable	Leftover var after refactor	Use data source, remove variable
3	Git file too large	.terraform/ committed	git rm --cached, add to .gitignore
4	Aurora invalid username	Reserved master username	Change to non-reserved name
5	CloudWatch dashboard invalid	Missing region in widgets	Add region to every widget
6	GitHub Actions if error	secrets not allowed in if:	Use continue-on-error: true
7	Trivy scan failing	Upstream app vulnerabilities	Add .trivyignore with explanation
8	Docker build path error	Wrong COPY path for context	Change COPY web/ ./ to COPY . .
9	ECR 401 Unauthorized	Missing IAM ECR permissions	Attach ECR policy to IAM user
10	Docker push boolean error	Quoted true string	Use unquoted true
11	Container unhealthy	curl missing in Alpine	apk add --no-cache curl
12	ALB can't reach ECS	Empty security group source	Add ALB SG to ECS task SG rule
13	API can't reach Aurora	Empty security group source	Add API ECS SG to Aurora SG rule
14	Web app 500 on root	Wrong env var name API_URL	Rename API_URL to API_HOST
15	Aurora connection rejected	SSL not configured in Node	Add ssl: rejectUnauthorized: false
16	App in development mode	NODE_ENV not set in task def	Set NODE_ENV=production

Detailed Issue Log

Issue 1 — Terraform Variables Syntax Error

Error	Invalid character ";" on variables.tf A single-line block definition must end with a closing brace
Cause	Terraform does not support semicolons to separate arguments inside single-line variable blocks.
Fix	Expanded all single-line variable blocks into multi-line format: <pre># Before (invalid) variable "web_cpu" { type = number; default = 512 }</pre> # After (correct) <pre>variable "web_cpu" { type = number default = 512 }</pre>

Issue 2 — Undeclared Variable availability_zones

Error	Reference to undeclared input variable "availability_zones"
Cause	The networking module was refactored to use a data source instead of a hardcoded AZ list, but the module call in environments/prod/main.tf still passed the removed variable.
Fix	Replaced hardcoded AZ variable with AWS data source: <pre>data "aws_availability_zones" "available" { state = "available" }</pre> Removed availability_zones from networking variables.tf, module call, and environment variables.tf.

Issue 3 — .terraform Directory Committed to Git

Error	File terraform/environments/prod/.terraform/providers/.../terraform-provider-aws is 674.20 MB; this exceeds GitHub file size limit of 100.00 MB
Cause	The .terraform/ directory containing provider binaries was accidentally committed before .gitignore was configured.
Fix	<pre># Remove from git tracking git rm -r --cached terraform/environments/prod/.terraform</pre>

	<pre># Add to .gitignore .terraform/ *.tfstate *.tfstate.backup # Remove from git history git filter-branch --force --index-filter \ "git rm -r --cached --ignore-unmatch terraform/environments/prod/.terraform" \ --prune-empty --tag-name-filter cat -- --all # Force push git push origin main --force</pre>
--	--

Issue 4 — Aurora Invalid Master Username

Error	<code>InvalidParameterValue: Invalid master user name</code>
Cause	AWS reserves certain usernames for Aurora PostgreSQL including: postgres, admin, root, dbadmin, administrator, rdsadmin. The default value in variables.tf was dbadmin.
Fix	Changed db_username to a non-reserved value: <pre>variable "db_username" { description = "Database master username" type = string default = "appuser" # was "dbadmin" - reserved by AWS }</pre>

Issue 5 — CloudWatch Dashboard Missing Region

Error	<code>InvalidParameterInput: The dashboard body is invalid - Should have required property 'region' (15 validation errors)</code>
Cause	Every CloudWatch dashboard widget requires an explicit region property inside its properties block. The original Terraform code omitted this field from all 5 widgets.
Fix	Added region = "us-east-1" to all 5 widget property blocks: <pre>properties = { title = "ECS Web - CPU & Memory" region = "us-east-1" # added to every widget period = 60 ... }</pre>

Issue 6 — GitHub Actions Invalid if Condition for Secrets

Error	Unrecognized named-value: 'secrets'. Located at position 1 within expression: secrets.SNYK_TOKEN != ''
Cause	GitHub Actions does not allow referencing the secrets context directly inside if: conditional expressions.
Fix	Replaced conditional skip with continue-on-error: true for optional tools: <pre># Before (invalid) - name: Run Snyk scan if: \${{ secrets.SNYK_TOKEN != '' }} # After (correct) - name: Run Snyk scan continue-on-error: true env: SNYK_TOKEN: \${{ secrets.SNYK_TOKEN }}</pre>

Issue 7 — Trivy Scan Failing on Upstream Vulnerabilities

Error	api/package-lock.json — Total: 11 (HIGH: 11) web/package-lock.json — Total: 19 (HIGH: 16, CRITICAL: 3) Error: Process completed with exit code 1
Cause	Legacy dependencies in the upstream node-3tier-app2 application contained known vulnerabilities. The app code is not owned by this project so the dependencies cannot be updated directly.
Fix	Created .trivyignore at repo root listing all CVEs with explanation: <pre># Third-party upstream app — not our code to fix CVE-2017-20165 CVE-2024-45296 CVE-2024-52798 GHSA-4vmm-mhcq-4x9j CVE-2025-7783 CVE-2019-5413 # ... all affected CVEs listed</pre> Trivy still runs and reports — it no longer fails the pipeline on issues outside project control.

Issue 8 — Docker Build Context Wrong Path

Error	failed to compute cache key: "/web": not found
Cause	Build context was set to ./web but the Dockerfile used COPY web/ ./ which tried to find a web/ folder inside the context. Since the context IS the web folder, this path doesn't exist.

Fix

Updated COPY instruction in Dockerfiles to use `.` instead of the folder name:

```
# Before (wrong – context is already ./web)
```

```
COPY web/ ./
```

```
# After (correct)
```

```
COPY ..
```

Rule: whatever folder is passed as context becomes `/` for all COPY instructions.

Issue 9 — ECR Push 401 Unauthorized

Error	<code>unexpected status from HEAD request to ECR: 401 Unauthorized</code>
Cause	The IAM user used by GitHub Actions had no ECR push permissions. It could authenticate to ECR but not write images.
Fix	Attached AmazonEC2ContainerRegistryFullAccess policy to the IAM user: AWS Console → IAM → Users → your-user → Permissions → Add permissions → Attach policies directly → AmazonEC2ContainerRegistryFullAccess

Issue 10 — Docker Build Push Boolean Error

Error	<code>Input does not meet YAML 1.2 Core Schema specification: push Support boolean input list: true True TRUE false False FALSE</code>
Cause	The push: field value was wrapped in quotes making it a YAML string instead of a boolean, which docker/build-push-action rejects.
Fix	Removed quotes around the boolean value: <pre># Before (invalid - string not boolean) push: "true" # After (correct) push: true</pre>

Issue 11 — ECS Container Unhealthy — curl Not Found

Error	<code>Container health status: UNHEALTHY Health check: curl -f http://localhost:3000/health exit 1</code>
Cause	The Alpine Linux base image used in the Dockerfile does not include curl by default. The ECS health check command failed because the binary was not present.
Fix	Added curl installation to both web and api Dockerfiles: RUN apk add --no-cache curl Also increased startPeriod to give app more startup time: <pre>healthCheck = { command = ["CMD-SHELL", "curl -f http://localhost:3000/health exit 1"] interval = 30 timeout = 10 retries = 3 startPeriod = 120 # increased from 60</pre>

```
}

```


Issue 12 — ALB Cannot Reach ECS Tasks — Empty Security Group Source

Error	App returns healthy inside container but 502/timeout through ALB
Cause	The ECS task security group had a rule for port 3000 but the UserIdGroupPairs array was empty. No source security group was specified so no traffic was actually permitted — the rule existed but allowed nobody.
Fix	Immediate fix via AWS CLI: <pre>aws ec2 authorize-security-group-ingress \ --group-id <ECS-TASK-SG-ID> \ --protocol tcp \ --port 3000 \ --source-group <ALB-SG-ID> \ --region us-east-1</pre> Permanent fix in modules/ecs/main.tf: <pre>resource "aws_security_group" "web_ecs" { ingress { from_port = 3000 to_port = 3000 protocol = "tcp" security_groups = [aws_security_group.web_alb.id] } }</pre>

Issue 13 — API Cannot Reach Aurora — Empty Security Group Source

Error	wget: error getting response: Resource temporarily unavailable (connection to Aurora port 5432 hangs)
Cause	Same empty UserIdGroupPairs issue as Issue 12, but on the Aurora security group. The rule for port 5432 existed but the API ECS security group was not set as the source.
Fix	<pre># Get security group IDs API_SG=\$(aws ecs describe-services \ --cluster node3tier-prod-cluster \ --services node3tier-prod-api-svc \ --query 'services[0].networkConfiguration.awsvpcConfiguration.securityGroups[0]' \ \ --output text) AURORA_SG=\$(aws rds describe-db-clusters \ --db-cluster-identifier node3tier-prod-aurora-cluster \ --query 'DBClusters[0].VpcSecurityGroups[0].VpcSecurityGroupId' \ --output text) # Add correct rule aws ec2 authorize-security-group-ingress \</pre>

```
--group-id $AURORA_SG \  
--protocol tcp \  
--port 5432 \  
--source-group $API_SG \  
--region us-east-1
```

Issue 14 — Web App 500 — Wrong Environment Variable Name

Error	error running request to http://.../api/status (web app error handler triggered)
Cause	The web app reads process.env.API_HOST to construct the API URL. The ECS task definition was setting API_URL instead. API_HOST was empty at runtime causing every request to the API to fail silently.
Fix	Updated ECS task definition in modules/ecs/main.tf: <pre>environment = [{ name = "NODE_ENV", value = "production" }, { name = "PORT", value = toString(var.web_port) }, { name = "API_HOST", value = "http://\${aws_lb.api.dns_name}" } # was API_URL - app reads API_HOST]</pre> Applied with: terraform apply + force new deployment

Issue 15 — Aurora Rejecting Unencrypted Connections

Error	no pg_hba.conf entry for host "10.0.10.23", user "appuser", database "appdb", no encryption
Cause	Aurora PostgreSQL enforces SSL connections by default. The Node.js pg library connects without SSL unless explicitly configured. Manual psql connections worked because psql enables SSL automatically.
Fix	Added SSL configuration to the connection object in api/app.js: <pre>const conString = { user: process.env.DBUSER, database: process.env.DB, password: process.env.DBPASS, host: process.env.DBHOST, port: process.env.DBPORT, ssl: { rejectUnauthorized: false // Aurora uses AWS managed certificate } };</pre> Committed and pushed — pipeline deployed new image automatically.

Issue 16 — App Running in Development Mode in Production

Error	res.render({...}) crash in error handler — every error returns 500 with no detail
Cause	NODE_ENV was not explicitly set in the ECS task definition so it defaulted to development. The development error handler in the upstream app code calls res.render() with an object instead of a template name string — a bug in the

	upstream code that only surfaces in development mode.
Fix	Explicitly set <code>NODE_ENV</code> in ECS task definition: <pre>environment = [{ name = "NODE_ENV", value = "production" }, ...]</pre> In production mode the app uses <code>res.json()</code> in the error handler which works correctly.

Lessons Learned

Infrastructure & Security Groups

- Always verify security group `UserIdGroupPairs` after Terraform apply — an empty array means the rule allows nobody despite appearing to exist.
- Run `aws elbv2 describe-target-health` after every ECS deployment to confirm ALB can reach targets before testing through CloudFront.
- Test connectivity at each layer independently: container → ALB → CloudFront, rather than jumping straight to the public URL.

Application Configuration

- Always match environment variable names exactly to what the app reads — check the source code before naming task definition env vars.
- Set `NODE_ENV=production` explicitly in every production ECS task definition. Never rely on defaults.
- Aurora PostgreSQL requires SSL. Always add `ssl: { rejectUnauthorized: false }` to the Node.js pg connection config when connecting to Aurora.

CI/CD & Docker

- Add `.terraform/`, `*.tfstate`, and `node_modules/` to `.gitignore` before the first commit — retroactively removing large files from history requires force-pushing.
- The Docker build context determines what `COPY` can access. If context is `./web` then `COPY . .` copies the web folder contents, not `COPY web/ ./`.
- Use `continue-on-error: true` for optional security tools (Snyk, SonarQube) so missing tokens don't block the pipeline.
- Use `.trivyignore` with explanatory comments for vulnerabilities in upstream code you don't own — this documents a conscious decision rather than a blind bypass.

Debugging Approach

- CloudWatch Logs are the first stop for container errors — use `aws logs tail /ecs/... --follow` while hitting the endpoint.
- ECS `execute-command` (exec into container) is essential for verifying env vars, testing connectivity, and inspecting the actual running code.
- The error shown to the user (e.g. 500) is often different from the root cause logged server-side — always check logs before assuming the failure location.

End of Document